Assignment 4, MSB 105, 2025

Karin Liang

Pakker

```r
library(tidyverse)
library(readxl)
library(restatapi)
library(DescTools)
library(ggrepel)
library(flextable)
library(modelr)
library(plm)
library(broom)
library(sandwich)
library(dplyr)
library(tidyr)
library(ggplot2)
```

Toc.eurostat

```r
# xml skal ha mer detaljert info
# toc_xml <- get_eurostat_toc()
# tekstversjonen har trolig nok info for vårt formål

toc_txt <- get_eurostat_toc(mode = "txt")
```

GDP NUTS 3

```
gdp_tabs <- toc_txt |>
# Regex AND external to regex
   filter(
     str_detect(
       string = title,
       # For å matche både små og store bokstaver
       pattern = '[Gg][Dd][Pp]'
       # AND vha. &
       ) &
     str_detect(
       string = title,
       # For å matche både små og store bokstaver og
       # space eller ikke før 3
       pattern = '[Nn][Uu][Tt][Ss]\\s*3'
       )
     ) |>
  select(title, code)
```

```
gdp_tabs |>
  select(title, code) |>
  flextable()  |>
  width(1, width = 3.5) |>
  width(2, width = 1.5)
```

title	code
Average annual population to calculate regional GDP data (thousand persons) by NUTS 3 region	nama_10r_3popgdp
Gross domestic product (GDP) at current market prices by NUTS 3 region	nama_10r_3gdp

```
# description nama_10r_3gdp
dsd_gdp <- get_eurostat_dsd("nama_10r_3gdp")
```

```
dsd_gdp |>
  filter(concept %in% c('freq', 'unit')) |>
  flextable()  |>
```

```
  width(j = 1, width = 1) |>
  width(j = 2, width = 2) |>
  width(j = 3, width = 2)
```

concept	code	name
freq	A	Annual
unit	MIO_EUR	Million euro
unit	EUR_HAB	Euro per inhabitant
unit	EUR_HAB_EU27_2020	Euro per inhabitant in percentage of the EU27 (from 2020) average
unit	MIO_NAC	Million units of national currency
unit	MIO_PPS_EU27_2020	Million purchasing power standards (PPS, EU27 from 2020)
unit	PPS_EU27_2020_HAB	Purchasing power standard (PPS, EU27 from 2020), per inhabitant
unit	PPS_HAB_EU27_2020	Purchasing power standard (PPS, EU27 from 2020), per inhabitant in percentage of the EU27 (from 2020) average

```
dsd_gdp |>
  filter(concept %in% c('geo')) |>
  head(n = 10) |>
  flextable()  |>
  width(j = 1, width = 1) |>
  width(j = 2, width = 2) |>
  width(j = 3, width = 2)
```

concept	code	name
geo	EU27_2020	European Union - 27 countries (from 2020)
geo	BE	Belgium
geo	BE1	Région de Bruxelles-Capitale/Brussels Hoofdstedelijk Gewest
geo	BE10	Région de Bruxelles-Capitale/Brussels Hoofdstedelijk Gewest
geo	BE100	Arr. de Bruxelles-Capitale/Arr. Brussel-Hoofdstad
geo	BE2	Vlaams Gewest
geo	BE21	Prov. Antwerpen
geo	BE211	Arr. Antwerpen
geo	BE212	Arr. Mechelen
geo	BE213	Arr. Turnhout

```
# Gross domestic product (GDP) at current market prices by NUTS 3 regions
# id: nama_10r_3gdp
# Vi velger å hente samtlige soner for så å filtrere ut de få vi ikke trenger
gdp <- get_eurostat_data(
  id = "nama_10r_3gdp",
  filters = list(
    # neste linje viser hvordan vi kunne ha hentet ut data
    # for spesifiserte land
    # geo = c("AT", "DE", "DK", "FR"),
    nuts_level = "3",
    unit = "MIO_PPS_EU27_2020"
  ),
  exact_match = FALSE,
  date_filter = 2000:2023,
  stringsAsFactors = FALSE
  ) |>
  mutate(
    gdp_n3 = 1000000 * values
```

```r
  ) |>
  select(-c(unit, values)) |>
  # Vil bare ha NUTS 3 nivå (5 karakterer). Vil aggregere selv til NUTS2,
  # NUTS1 og NUTSc
  filter(str_length(geo) == 5) |>
  as_tibble()
```

```r
dim(gdp)
```

```
[1] 30058     3
```

```r
gdp
```

```
# A tibble: 30,058 x 3
   geo   time     gdp_n3
   <chr> <chr>     <dbl>
 1 AL011 2008  551130000
 2 AL011 2009  582160000
 3 AL011 2010  664070000
 4 AL011 2011  631170000
 5 AL011 2012  717600000
 6 AL011 2013  696860000
 7 AL011 2014  735600000
 8 AL011 2015  788630000
 9 AL011 2016  801980000
10 AL011 2017  800660000
# i 30,048 more rows
```

```r
gdp |>
  filter(geo == "IE053") |>
  print(n = 25)
```

```
# A tibble: 21 x 3
   geo   time        gdp_n3
   <chr> <chr>        <dbl>
 1 IE053 2000   15837300000
 2 IE053 2001   17506250000
 3 IE053 2002   19395440000
 4 IE053 2003   19687190000
 5 IE053 2004   21000450000
```

```
 6 IE053 2005   21776750000
 7 IE053 2006   24081640000
 8 IE053 2007   26086890000
 9 IE053 2008   22705550000
10 IE053 2009   24012370000
11 IE053 2010   24085200000
12 IE053 2011   26235110000
13 IE053 2012   24346250000
14 IE053 2013   23345250000
15 IE053 2014   25127580000
16 IE053 2018   73687140000
17 IE053 2019   71965850000
18 IE053 2020   75581570000
19 IE053 2021   99064470000
20 IE053 2022  122163400000
21 IE053 2023  103989840000
```

```
ie_data <- tibble(
  geo = c("IE053", "IE053", "IE053"),
  time = c("2015", "2016", "2017"),
  gdp_n3 = c(NA, NA, NA)
)
```

```
gdp <- rbind(gdp, ie_data)
```

```
gdp <- gdp |>
  arrange(geo, time) |>
  mutate(
    gdp_n3 = zoo::na.approx(gdp_n3)
  )
```

```
gdp |>
  filter(geo == "IE053") |>
  print(n = 25)
```

```
# A tibble: 24 x 3
   geo   time        gdp_n3
   <chr> <chr>        <dbl>
 1 IE053 2000   15837300000
 2 IE053 2001   17506250000
 3 IE053 2002   19395440000
```

```
 4 IE053 2003   19687190000
 5 IE053 2004   21000450000
 6 IE053 2005   21776750000
 7 IE053 2006   24081640000
 8 IE053 2007   26086890000
 9 IE053 2008   22705550000
10 IE053 2009   24012370000
11 IE053 2010   24085200000
12 IE053 2011   26235110000
13 IE053 2012   24346250000
14 IE053 2013   23345250000
15 IE053 2014   25127580000
16 IE053 2015   37267470000
17 IE053 2016   49407360000
18 IE053 2017   61547250000
19 IE053 2018   73687140000
20 IE053 2019   71965850000
21 IE053 2020   75581570000
22 IE053 2021   99064470000
23 IE053 2022  122163400000
24 IE053 2023  103989840000
```

Population

Oppgave 1

```
# Søk i toc_txt etter tabeller som inneholder både population og NUTS 3
# Vi bruker regexp for å dekke både små og store bokstaver og ulike skrivemåter

pop_tabs <- toc_txt |>
  filter(
    str_detect(
      string  = title,
      pattern = "[Pp]opulation"
    ) &
      str_detect(
        string  = title,
        pattern = "[Nn][Uu][Tt][Ss]\\s*3"
      )
```

```
  ) |>
  select(title, code)
```

```
pop_tabs |>
  flextable() |>
  width(1, width = 5) |>
  width(2, width = 2)
```

title	code
Population density by NUTS 3 region	demo_r_d3dens
Population on 1 January by age group, sex and NUTS 3 region	demo_r_pjangrp3
Population on 1 January by broad age group, sex and NUTS 3 region	demo_r_pjanaggr3
Population structure indicators by NUTS 3 region	demo_r_pjanind3
Population change - Demographic balance and crude rates at regional level (NUTS 3)	demo_r_gind3
Population by single year of age and NUTS 3 region	cens_11ag_r3
Population by marital status and NUTS 3 region	cens_11ms_r3
Population by family status and NUTS 3 region	cens_11fs_r3
Population by sex, citizenship and NUTS 3 region	cens_01rsctz
Population by sex, age group, current activity status and NUTS 3 region	cens_01rapop
Total and active population by sex, age, employment status, residence one year prior to the census and NUTS 3 region	cens_01ramigr
Population by sex, age group, educational attainment level, current activity status and NUTS 3 region	cens_01rews
Population by sex, age group, household status and NUTS 3 region	cens_01rhtype
Population by sex, age group, size of household and NUTS 3 region	cens_01rhsize
Average annual population to calculate regional GDP data (thousand persons) by NUTS 3 region	nama_10r_3popgdp
Population by country of citizenship, age groups and NUTS 3 region	cens_21ctz_r3
Population by country of citizenship, age groups, family status and NUTS 3 region	cens_21ctzf_r3

title	code
Population by country of citizenship, age groups, type of housing arrangements and NUTS 3 region	cens_21ctzha_r3
Population by country of birth, age groups and NUTS 3 region	cens_21cob_r3
Population by country of birth, age groups, household status and NUTS 3 region	cens_21cobhs_r3
Population by country of birth, age groups, type of housing arrangements and NUTS 3 region	cens_21cobha_r3
Population by marital status, broad age groups and NUTS 3 region	cens_21m_r3
Population by family status, broad age groups and NUTS 3 region	cens_21f_r3
Population by size of the locality, age groups and NUTS 3 region	cens_21l_r3
Population by size of the locality, housing arrangements and NUTS 3 region	cens_21lha_r3
Population by year of arrival in the country since 2010, age groups, groups of country of birth and NUTS 3 region	cens_21argc_r3
Population by year of arrival in the country, age groups, family status and NUTS 3 region	cens_21arf_r3
Population with Ukrainian citizenship by 5-year age group and NUTS 3 region	cens_21ua_a5r3
Population with Ukrainian citizenship by age and NUTS 3 region	cens_21ua_ar3
Population with Ukrainian citizenship by 5-year age group, marital status and NUTS 3 region	cens_21ua_msr3
Population on 1st January by age, sex, type of projection and NUTS 3 region	proj_19rp3

2

```
# Last ned Data Structure Definition (DSD) for nama_10r_3popgdp

dsd_pop <- get_eurostat_dsd("nama_10r_3popgdp")
```

```
dsd_pop |>
  filter(concept %in% c("freq", "unit", "geo")) |>
```

```r
  head(n = 45) |>
  flextable() |>
  fontsize(size = 7, part = "all") |>
  line_spacing(space = 0.1) |>
  autofit()
```

concept	code	name
freq	A	Annual
unit	THS	Thousand
geo	EU27_2020	European Union - 27 countries (from 2020)
geo	BE	Belgium
geo	BE1	Région de Bruxelles-Capitale/Brussels Hoofdstedelijk Gewest
geo	BE10	Région de Bruxelles-Capitale/Brussels Hoofdstedelijk Gewest
geo	BE100	Arr. de Bruxelles-Capitale/Arr. Brussel-Hoofdstad
geo	BE2	Vlaams Gewest
geo	BE21	Prov. Antwerpen
geo	BE211	Arr. Antwerpen
geo	BE212	Arr. Mechelen
geo	BE213	Arr. Turnhout
geo	BE22	Prov. Limburg (BE)
geo	BE223	Arr. Tongeren
geo	BE224	Arr. Hasselt
geo	BE225	Arr. Maaseik
geo	BE23	Prov. Oost-Vlaanderen
geo	BE231	Arr. Aalst
geo	BE232	Arr. Dendermonde
geo	BE233	Arr. Eeklo
geo	BE234	Arr. Gent
geo	BE235	Arr. Oudenaarde
geo	BE236	Arr. Sint-Niklaas
geo	BE24	Prov. Vlaams-Brabant
geo	BE241	Arr. Halle-Vilvoorde
geo	BE242	Arr. Leuven
geo	BE25	Prov. West-Vlaanderen
geo	BE251	Arr. Brugge
geo	BE252	Arr. Diksmuide
geo	BE253	Arr. Ieper
geo	BE254	Arr. Kortrijk
geo	BE255	Arr. Oostende
geo	BE256	Arr. Roeselare
geo	BE257	Arr. Tielt
geo	BE258	Arr. Veurne
geo	BE3	Région wallonne
geo	BE31	Prov. Brabant wallon
geo	BE310	Arr. Nivelles
geo	BE32	Prov. Hainaut
geo	BE323	Arr. Mons
geo	BE328	Arr. Tournai-Mouscron
geo	BE329	Arr. La Louvière
geo	BE32A	Arr. Ath
geo	BE32B	Arr. Charleroi
geo	BE32C	Arr. Soignies

```r
# Bruk DSD-informasjonen til å laste ned pop-data på NUTS3-nivå
pop <- get_eurostat_data(
```

```
  id = "nama_10r_3popgdp",
  filters = list(
    freq = "A",       # Annual
    unit = "THS"     # Thousand (jf. DSD-tabellen over)
  ),
  exact_match = FALSE,
  date_filter = 2000:2023,
  stringsAsFactors = FALSE
) |>
  mutate(pop_n3 = 1000 * values) |>
  select(geo, time, pop_n3) |>
  filter(str_length(geo) == 5) |>
  as_tibble()
```

```
dim(pop)
```

```
[1] 30038     3
```

```
pop |>
  filter(str_sub(geo, 1,2) == "AL") |>
  nrow()
```

```
[1] 252
```

```
# ag: ikke så smart å printe alle 30000
pop |>
  head(n = 126) %>%
  {
    list(
      p1 = slice(., 1:42),
      p2 = slice(., 43:84),
      p3 = slice(., 85:126)
    )
  } |>
  bind_cols() |>
  flextable() |>
  set_header_labels(
    geo...1 = "NUTS3",
    time...2 = "År",
    pop_n3...3 = "Befolkning\nNUTS3",
```

```r
    geo...4 = "NUTS3",
    time...5 = "År",
    pop_n3...6 = "Befolkning\nNUTS3",
    geo...7 = "NUTS3",
    time...8 = "År",
    pop_n3...9 = "Befolkning\nNUTS3"
  ) |>
  line_spacing(space = 0.1)  |>
  fontsize(size = 9, part = "all") |>
  align(j = c(3, 6, 9), align = "left", part = "all") |>
  autofit()
```

Table 6: First 126 observations in pop

NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3
AL011	2001	186,720	AL013	2001	109,440	AL015	2001	254,300
AL011	2002	182,380	AL013	2002	107,150	AL015	2002	251,190
AL011	2003	178,710	AL013	2003	105,580	AL015	2003	248,800
AL011	2004	174,710	AL013	2004	104,020	AL015	2004	246,860
AL011	2005	170,370	AL013	2005	102,130	AL015	2005	244,520
AL011	2006	165,610	AL013	2006	99,930	AL015	2006	241,500
AL011	2007	160,540	AL013	2007	97,540	AL015	2007	237,710
AL011	2008	155,390	AL013	2008	94,970	AL015	2008	233,840
AL011	2009	150,430	AL013	2009	92,430	AL015	2009	230,580
AL011	2010	146,140	AL013	2010	90,310	AL015	2010	227,350
AL011	2011	142,580	AL013	2011	88,580	AL015	2011	224,090
AL011	2012	139,340	AL013	2012	86,930	AL015	2012	221,090
AL011	2013	136,020	AL013	2013	85,180	AL015	2013	218,010
AL011	2014	132,690	AL013	2014	83,370	AL015	2014	214,820
AL011	2015	130,050	AL013	2015	81,850	AL015	2015	211,660
AL011	2016	127,320	AL013	2016	80,420	AL015	2016	209,050
AL011	2017	123,290	AL013	2017	78,480	AL015	2017	206,460

NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3
AL011	2018	119,970	AL013	2018	76,990	AL015	2018	203,940
AL011	2019	117,410	AL013	2019	76,010	AL015	2019	201,450
AL011	2020	114,770	AL013	2020	74,910	AL015	2020	198,590
AL011	2021	111,630	AL013	2021	73,580	AL015	2021	195,090
AL012	2001	246,560	AL014	2001	158,490	AL021	2001	359,510
AL012	2002	249,440	AL014	2002	157,160	AL021	2002	353,950
AL012	2003	251,450	AL014	2003	155,230	AL021	2003	348,310
AL012	2004	253,310	AL014	2004	153,280	AL021	2004	342,610
AL012	2005	255,540	AL014	2005	151,230	AL021	2005	336,550
AL012	2006	257,710	AL014	2006	148,840	AL021	2006	330,410
AL012	2007	259,480	AL014	2007	146,410	AL021	2007	324,140
AL012	2008	261,630	AL014	2008	144,060	AL021	2008	317,930
AL012	2009	264,250	AL014	2009	141,810	AL021	2009	312,210
AL012	2010	267,710	AL014	2010	140,250	AL021	2010	308,300
AL012	2011	270,870	AL014	2011	138,750	AL021	2011	304,900
AL012	2012	272,990	AL014	2012	136,740	AL021	2012	300,930
AL012	2013	275,040	AL014	2013	134,710	AL021	2013	296,960
AL012	2014	277,010	AL014	2014	132,790	AL021	2014	292,800
AL012	2015	279,090	AL014	2015	131,040	AL021	2015	289,140
AL012	2016	282,510	AL014	2016	129,640	AL021	2016	285,710
AL012	2017	287,230	AL014	2017	127,910	AL021	2017	281,190
AL012	2018	289,870	AL014	2018	126,000	AL021	2018	276,770
AL012	2019	290,410	AL014	2019	123,950	AL021	2019	272,530
AL012	2020	291,360	AL014	2020	121,690	AL021	2020	268,160
AL012	2021	291,680	AL014	2021	118,980	AL021	2021	262,680

```r
# ag: ikke så smart å printe alle 30000. Tror det holder lenge
# med disse 252 observasjonene
pop |>
  _[127:252,] %>%
  {
    list(
      p1 = slice(., 1:42),
      p2 = slice(., 43:84),
      p3 = slice(., 85:126)
    )
  } |>
  bind_cols() |>
  flextable() |>
  set_header_labels(
    geo...1 = "NUTS3",
    time...2 = "År",
    pop_n3...3 = "Befolkning\nNUTS3",
    geo...4 = "NUTS3",
    time...5 = "År",
    pop_n3...6 = "Befolkning\nNUTS3",
    geo...7 = "NUTS3",
    time...8 = "År",
    pop_n3...9 = "Befolkning\nNUTS3"
  ) |>
  line_spacing(space = 0.1)  |>
  fontsize(size = 9, part = "all") |>
  align(j = c(3, 6, 9), align = "left", part = "all") |>
  autofit()
```

Table 7: Next 126 observations in pop (the rest of Albania).

NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3
AL022	2001	607,130	AL032	2001	379,850	AL034	2001	263,230
AL022	2002	625,630	AL032	2002	375,470	AL034	2002	260,050
AL022	2003	642,240	AL032	2003	370,570	AL034	2003	256,910
AL022	2004	659,090	AL032	2004	365,070	AL034	2004	253,590
AL022	2005	675,670	AL032	2005	358,930	AL034	2005	250,250

NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3
AL022	2006	692,050	AL032	2006	352,670	AL034	2006	246,590
AL022	2007	707,750	AL032	2007	346,110	AL034	2007	242,730
AL022	2008	723,170	AL032	2008	339,500	AL034	2008	238,890
AL022	2009	739,090	AL032	2009	333,370	AL034	2009	235,360
AL022	2010	755,370	AL032	2010	327,620	AL034	2010	232,040
AL022	2011	772,230	AL032	2011	322,830	AL034	2011	229,190
AL022	2012	789,570	AL032	2012	318,700	AL034	2012	226,880
AL022	2013	807,350	AL032	2013	314,420	AL034	2013	224,320
AL022	2014	825,230	AL032	2014	310,120	AL034	2014	221,610
AL022	2015	838,610	AL032	2015	306,560	AL034	2015	218,800
AL022	2016	852,690	AL032	2016	303,810	AL034	2016	215,870
AL022	2017	873,160	AL032	2017	300,320	AL034	2017	212,250
AL022	2018	889,570	AL032	2018	296,450	AL034	2018	209,040
AL022	2019	900,660	AL032	2019	292,320	AL034	2019	206,360
AL022	2020	909,170	AL032	2020	287,950	AL034	2020	203,510
AL022	2021	915,850	AL032	2021	282,210	AL034	2021	199,750
AL031	2001	190,830	AL033	2001	111,120	AL035	2001	193,010
AL031	2002	187,020	AL033	2002	108,110	AL035	2002	193,460
AL031	2003	183,110	AL033	2003	105,050	AL035	2003	193,660
AL031	2004	179,080	AL033	2004	101,790	AL035	2004	193,540
AL031	2005	174,690	AL033	2005	98,410	AL035	2005	193,200
AL031	2006	169,910	AL033	2006	94,830	AL035	2006	192,520
AL031	2007	164,970	AL033	2007	91,180	AL035	2007	191,460
AL031	2008	160,060	AL033	2008	87,610	AL035	2008	190,280
AL031	2009	155,340	AL033	2009	84,060	AL035	2009	188,600
AL031	2010	151,370	AL033	2010	80,320	AL035	2010	186,250
AL031	2011	148,200	AL033	2011	77,220	AL035	2011	185,750

NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3	NUTS3	År	Befolkning NUTS3
AL031	2012	145,300	AL033	2012	74,910	AL035	2012	187,040
AL031	2013	142,480	AL033	2013	72,650	AL035	2013	187,960
AL031	2014	139,740	AL033	2014	70,570	AL035	2014	188,370
AL031	2015	136,900	AL033	2015	68,780	AL035	2015	188,220
AL031	2016	133,690	AL033	2016	66,990	AL035	2016	188,410
AL031	2017	129,690	AL033	2017	64,450	AL035	2017	189,040
AL031	2018	126,300	AL033	2018	62,190	AL035	2018	189,300
AL031	2019	123,580	AL033	2019	60,400	AL035	2019	189,120
AL031	2020	120,730	AL033	2020	58,710	AL035	2020	188,300
AL031	2021	117,250	AL033	2021	56,660	AL035	2021	186,320

Oppgave 3

```r
# Slå sammen GDP- og befolkningstabellene
# Viktig: gdp skal være venstre tabell

gdp_pop <- gdp |>
  left_join(pop, by = c("geo", "time"))
```

```r
eu_data <- gdp_pop %>%
  # Trenger ikke ZZ sonene som er en slag oppsamlingssone
  # for ikke fordelte verdier
  filter(!str_sub(geo, 3, 4) == "ZZ") |>
  # Drop the EFTA countries Switzerland and Norway
  filter(!str_sub(geo, 1, 2) %in% c("CH", "NO")) |>
  # Drop the EU countries Netherlands and Portugal
  filter(!str_sub(geo, 1, 2) %in% c("NL", "PT")) |>
  # Drop candidate country Monte Negro because of data
  filter(!str_sub(geo, 1, 2) %in% c("ME")) |>
  # Drop a region of France in the Indian Ocean (Outre Mer); Mayotte
  # because of missing data
  filter(!geo == "FRY50") |>
  # note that a few countries will have missing data for
```

```r
  # some years at the start of the period
  filter(time > 1999 & time < 2023)
```

```r
eu_data <- eu_data |>
  mutate(
  gdp_pc_n3 = gdp_n3 / pop_n3
)
```

```r
dim(eu_data)
```

```
[1] 27584     5
```

```r
dim(
  eu_data |>
    filter(is.na(gdp_pc_n3))
)
```

```
[1] 0 5
```

Oppgave 4

```r
# Endre geo til n3 og lag variablene n2, n1 og nc

eu_data <- eu_data |>
  # Endrer navnet fra geo til n3
  rename(n3 = geo) |>

  # Lager nye variabler basert på n3-koden
  mutate(
    n2 = str_sub(n3, 1, 4),   # NUTS2: første 4 tegn
    n1 = str_sub(n3, 1, 3),   # NUTS1: første 3 tegn
    nc = str_sub(n3, 1, 2)    # Landskode: første 2 tegn
  )
```

```r
eu_data |>
  select(n3, n2, n1, nc) |>
  head(10)
```

```
# A tibble: 10 x 4
   n3    n2    n1    nc   
   <chr> <chr> <chr> <chr>
 1 AL011 AL01  AL0   AL   
 2 AL011 AL01  AL0   AL   
 3 AL011 AL01  AL0   AL   
 4 AL011 AL01  AL0   AL   
 5 AL011 AL01  AL0   AL   
 6 AL011 AL01  AL0   AL   
 7 AL011 AL01  AL0   AL   
 8 AL011 AL01  AL0   AL   
 9 AL011 AL01  AL0   AL   
10 AL011 AL01  AL0   AL   
```

Oppgave 5

```r
# Sjekk om noen NUTS3-soner har pop_n3 lik 0, og sett disse til NA

# Sjekker om det finnes verdier som er 0
sum(eu_data$pop_n3 == 0, na.rm = TRUE)
```

```
[1] 0
```

```r
# Setter pop_n3 = 0 til NA dersom slike finnes
eu_data <- eu_data |>
  mutate(
    pop_n3 = ifelse(pop_n3 == 0, NA, pop_n3)
  )
```

Oppgave 6

```r
# Teller antall unike NUTS3-soner i hvert land
nuts3_per_land <- eu_data |>
  group_by(nc) |>
  summarise(Antall = n_distinct(n3)) |>
  arrange(nc)

# Teller hvor mange land (nc) vi har totalt
```

```r
n_lands <- nuts3_per_land |> summarise(n = n()) |> pull(n)

# Lager flextable og legger til en footer med "n: xx"
flextable_nuts3 <- nuts3_per_land |>
  flextable() |>
  width(j = 1, width = 1) |>
  width(j = 2, width = 1) |>
  add_footer_lines(values = paste0("n: ", n_lands)) |>
  align(align = "left", part = "footer")
```

```r
# ag: Forslag til hvordan man kan lage en mer kompakt tabell
nuts3_per_land  |>
  arrange(nc)  %>%
  bind_rows(
    # ag: må legge til en tibble med 3 rekker med NA for å
    # få 32 rekker i tibble
    as_tibble(
      matrix(
        rep(NA, 6),
        nrow = 3,
        dimnames = list(NULL, c("nc", "Antall")
                        )
        )
      )
  ) %>%
  # ag: splitter opp i liste med 4 objekter
  {
    list(
      p1 = slice(., 1:8),
      p2 = slice(., 9:16),
      p3 = slice(., 17:24),
      p4 = slice(., 25:32)
    )
  } |>
  # ag: binder de fire sammen kolonnevis
  bind_cols() |>
  flextable() |>
  # ag: flextable krever unike variabelnavn, men vi kan
  # ha like heading labels
  set_header_labels(
    nc...1 = "Land\nkode",
    Antall...2 = "Antall",
```

```r
    nc...3 = "Land\nkode",
    Antall...4 = "Antall",
    nc...5 = "Land\nkode",
    Antall...6 = "Antall",
    nc...7 = "Land\nkode",
    Antall...8 = "Antall"
  ) |>
  line_spacing(space = 0.5, part = "body")  |>
  align(align = "left", part = "all") |>
  autofit()
```

Land kode	Antall	Land kode	Antall	Land kode	Antall	Land kode	Antall
AL	12	EE	5	IT	107	RS	25
AT	35	EL	52	LT	10	SE	21
BE	44	ES	59	LU	1	SI	12
BG	28	FI	19	LV	5	SK	8
CY	1	FR	100	MK	8	TR	81
CZ	14	HR	21	MT	2		
DE	400	HU	20	PL	73		
DK	11	IE	8	RO	42		

Oppgave 7

```r
# Sjekk summary for gdp_pc_n3
# Viser et standard summary for variabelen gdp_pc_n3
summary(eu_data$gdp_pc_n3)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
   2214   14994   21144   22782   27952  180416 
```

```r
# Minste verdi
min_gdp_pc <- min(eu_data$gdp_pc_n3, na.rm = TRUE)
min_gdp_pc
```

```
[1] 2214.485
```

```r
# Største verdi
max_gdp_pc <- max(eu_data$gdp_pc_n3, na.rm = TRUE)
max_gdp_pc
```

```
[1] 180415.8
```

```r
# Antall NA-verdier i gdp_pc_n3
sum(is.na(eu_data$gdp_pc_n3))
```

```
[1] 0
```

Datasettet gdp__pc__n3 har 0 NAV-verdie, og den minste verdien er 2214, mens den største verdien er 180416.

Oppgave 8

```r
# Legg til variabelen nc_name basert på landskode (nc)

eu_data <- eu_data |>
  mutate(
    nc_name = case_when(
      nc == "AL" ~ "Albania",
      nc == "AT" ~ "Østerrike",
      nc == "BE" ~ "Belgia",
      nc == "BG" ~ "Bulgaria",
      nc == "CY" ~ "Kypros",
      nc == "CZ" ~ "Tjekkia",
      nc == "DE" ~ "Tyskland",
      nc == "DK" ~ "Danmark",
      nc == "EE" ~ "Estland",
      nc == "EL" ~ "Hellas",
      nc == "ES" ~ "Spania",
      nc == "FI" ~ "Finland",
      nc == "FR" ~ "Frankrike",
      nc == "HR" ~ "Kroatia",
      nc == "HU" ~ "Ungarn",
      nc == "IE" ~ "Irland",
```

```
      nc == "IT" ~ "Italia",
      nc == "LT" ~ "Litauen",
      nc == "LU" ~ "Luxembourg",
      nc == "LV" ~ "Latvia",
      nc == "MK" ~ "Nord-Makedonia",
      nc == "MT" ~ "Malta",
      nc == "PL" ~ "Polen",
      nc == "RO" ~ "Romania",
      nc == "RS" ~ "Serbia",
      nc == "SE" ~ "Sverige",
      nc == "SI" ~ "Slovenia",
      nc == "SK" ~ "Slovakia",
      nc == "TR" ~ "Tyrkia",
      TRUE ~ NA_character_
    )
  )
```

```
# ag: Forslag til kompakt tabell for land og landkoder
eu_data |>
  select(nc_name, nc) |>
  distinct() |>
  arrange(nc)  %>%
  bind_rows(
    # ag: må legge til en tibble med 3 rekker med NA for å
    # få 32 rekker i tibble
    as_tibble(
      matrix(
        rep(NA, 6),
        nrow = 3,
        dimnames = list(NULL, c("nc_name", "nc")
                        )
        )
      )
  ) %>%
  # ag: splitter opp i liste med 4 objekter
  {
    list(
      p1 = slice(., 1:8),
      p2 = slice(., 9:16),
      p3 = slice(., 17:24),
      p4 = slice(., 25:32)
    )
```

```r
  } |>
  # ag: binder de fire sammen kolonnevis
  bind_cols() |>
  flextable() |>
  # ag: flextable krever unike variabelnavn, men vi kan
  # ha like heading labels
  set_header_labels(
    nc_name...1 = "Land",
    nc...2 = "Land\nkode",
    nc_name...3 = "Land",
    nc...4 = "Land\nkode",
    nc_name...5 = "Land",
    nc...6 = "Land\nkode",
    nc_name...7 = "Land",
    nc...8 = "Land\nkode"
  ) |>
  line_spacing(space = 0.5, part = "body")  |>
  align(align = "left", part = "all") |>
  autofit()
```

Table 9: Navn på land og tilhørende landkode.

Land	Land kode	Land	Land kode	Land	Land kode	Land	Land kode
Albania	AL	Estland	EE	Italia	IT	Serbia	RS
Østerrike	AT	Hellas	EL	Litauen	LT	Sverige	SE
Belgia	BE	Spania	ES	Luxembourg	LU	Slovenia	SI
Bulgaria	BG	Finland	FI	Latvia	LV	Slovakia	SK
Kypros	CY	Frankrike	FR	Nord-Makedonia	MK	Tyrkia	TR
Tjekkia	CZ	Kroatia	HR	Malta	MT		
Tyskland	DE	Ungarn	HU	Polen	PL		
Danmark	DK	Irland	IE	Romania	RO		

Beregning av Gini på NUTS2, NUTS1 og NUTSc nivå

Oppgave 9

```r
# Beregne Gini-koeffisienter på NUTS2 nivå

gini_n2 <- eu_data |>
  group_by(n2, time, n1, nc, nc_name) |>
  summarise(
    gini_n2 = DescTools::Gini(gdp_pc_n3, weights = pop_n3, na.rm = TRUE),
    pop_n2  = sum(pop_n3),
    gdp_n2  = sum(gdp_n3),
    gdp_pc_n2 = gdp_n2 / pop_n2,
    num_reg_n2 = n(),
    .groups = "drop"
  )

# Kort oversikt
gini_n2 |>
  select(gini_n2, num_reg_n2, gdp_n2, pop_n2, gdp_pc_n2) |>
  summary()
```

```
    gini_n2          num_reg_n2         gdp_n2              pop_n2
 Min.   :0.00038   Min.   : 1.000   Min.   :6.814e+08   Min.   :   25740
 1st Qu.:0.06753   1st Qu.: 2.000   1st Qu.:1.595e+10   1st Qu.:  992732
 Median :0.10893   Median : 4.000   Median :3.030e+10   Median : 1529210
 Mean   :0.12316   Mean   : 4.819   Mean   :4.679e+10   Mean   : 1947314
 3rd Qu.:0.16290   3rd Qu.: 6.000   3rd Qu.:5.388e+10   3rd Qu.: 2361818
 Max.   :0.47793   Max.   :23.000   Max.   :7.083e+11   Max.   :15874440
 NA's   :856
   gdp_pc_n2
 Min.   : 3157
 1st Qu.:15317
 Median :21839
 Mean   :23011
 3rd Qu.:28793
 Max.   :96746
```

Oppgave 10

```r
gini_n2 |>
  filter(gini_n2 < 0.001) |>
  arrange(num_reg_n2) |>
  select(nc_name, nc, n1, n2, time, gini_n2, num_reg_n2) |>
  print(n = 10)
```

```
# A tibble: 4 x 7
  nc_name  nc    n1    n2    time   gini_n2 num_reg_n2
  <chr>    <chr> <chr> <chr> <chr>    <dbl>      <int>
1 Danmark  DK    DK0   DK02  2019  0.000977          2
2 Italia   IT    ITF   ITF5  2006  0.000545          2
3 Polen    PL    PL4   PL43  2011  0.000854          2
4 Slovakia SK    SK0   SK03  2004  0.000379          2
```

Disse regionene består av svært få NUTS3-soner, og lav Gini skyldes derfor datastrukturen snarere enn reelle regionale forskjeller.

Oppgave 11

```r
# Beregner Gini-koeffisienter og økonomiske nøkkeltall på NUTS1 nivå
gini_n1 <- eu_data |>
  group_by(n1, time, nc, nc_name) |>
  summarise(
    gini_n1    = DescTools::Gini(gdp_pc_n3, weights = pop_n3, na.rm = TRUE),
    pop_n1     = sum(pop_n3),
    gdp_n1     = sum(gdp_n3),
    gdp_pc_n1 = gdp_n1 / pop_n1,
    num_reg_n1 = n(),
    .groups = "drop"
  )

# Summary
gini_n1 |>
  select(gini_n1, num_reg_n1, gdp_n1, pop_n1, gdp_pc_n1) |>
  summary() |>
  print(width = 76)
```

```
    gini_n1           num_reg_n1        gdp_n1                pop_n1
 Min.   :0.01601   Min.   : 1.00   Min.   :6.814e+08   Min.   :   25740
 1st Qu.:0.09123   1st Qu.: 6.00   1st Qu.:4.256e+10   1st Qu.: 2689490
 Median :0.13959   Median : 9.00   Median :7.888e+10   Median : 3934280
 Mean   :0.15364   Mean   :12.22   Mean   :1.187e+11   Mean   : 4938602
 3rd Qu.:0.18790   3rd Qu.:14.00   3rd Qu.:1.411e+11   3rd Qu.: 5992840
 Max.   :0.42934   Max.   :96.00   Max.   :7.287e+11   Max.   :18031860
 NA's   :177
   gdp_pc_n1
 Min.   : 3802
 1st Qu.:15750
 Median :22295
 Mean   :23523
 3rd Qu.:29340
 Max.   :90512
```

Oppgave 12

```r
# Beregner Gini-koeffisienter på landsnivå (NUTSc), basert på NUTS3-verdier
gini_nc <- eu_data |>
  group_by(nc, nc_name, time) |>
  summarise(
    gini_nc   = DescTools::Gini(gdp_pc_n3, weights = pop_n3, na.rm = TRUE),
    pop_nc    = sum(pop_n3),
    gdp_nc    = sum(gdp_n3),
    gdp_pc_nc = gdp_nc / pop_nc,
    num_reg_nc = n(),         # antall NUTS3 i landet
    .groups = "drop"
  )

# Summary
gini_nc |>
  select(gini_nc, num_reg_nc, gdp_nc, pop_nc, gdp_pc_nc) |>
  summary() |>
  print(width = 80)
```

```
    gini_nc          num_reg_nc         gdp_nc                pop_nc
 Min.   :0.1111   Min.   :  1.00   Min.   :5.892e+09   Min.   :  386200
 1st Qu.:0.1742   1st Qu.:  8.00   1st Qu.:4.350e+10   1st Qu.: 2810745
```

```
 Median :0.2094   Median : 20.00   Median :1.516e+11   Median : 6984230
 Mean   :0.2149   Mean   : 42.37   Mean   :4.114e+11   Mean   :17122006
 3rd Qu.:0.2553   3rd Qu.: 44.00   3rd Qu.:3.447e+11   3rd Qu.:11352985
 Max.   :0.3991   Max.   :400.00   Max.   :3.550e+12   Max.   :84979990
 NA's   :46
   gdp_pc_nc
 Min.   : 4854
 1st Qu.:15103
 Median :22224
 Mean   :23761
 3rd Qu.:29362
 Max.   :90512
```

«Nestete» datastrukturer

Oppgave 13

```
# Nest dataene på NUTS3 nivå
gini_n3_nest <- eu_data |>
  group_by(nc_name, nc) |>
  nest(.key = "NUTS3_data") |>
  ungroup()
```

Oppgave 14

```
gini_NUTS2_nest <- gini_n2 |>
  group_by(nc_name, nc) |>
  nest(.key = "NUTS2_data") |>
  ungroup()
```

Oppgave 15

```
gini_NUTS1_nest <- gini_n1 |>
  group_by(nc_name, nc) |>
  nest(.key = "NUTS1_data") |>
  ungroup()
```

Oppgave 16

```
gini_NUTSc_nest <- gini_nc |>
  group_by(nc_name, nc) |>
  nest(.key = "NUTSc_data") |>
  ungroup()
```

Oppgave 17

```
gini_NUTSeu_nest <- eu_data |>
  group_by(time) |>
  summarise(
    gini_eu = DescTools::Gini(gdp_pc_n3, weights = pop_n3, na.rm = TRUE),
    num_reg_eu = n(),
    .groups = "drop"
  )
```

Oppgave 18

```
gini_NUTSeu_nest <- gini_NUTSeu_nest |>
  mutate(time = as.numeric(time))
```

```
ggplot(gini_NUTSeu_nest, aes(x = time, y = gini_eu)) +
  geom_line() +
  theme_minimal() +
  labs(
    title = "Regional ulikhet i EU",
    x = "Year",
    y = "gini_eu"
  )
```

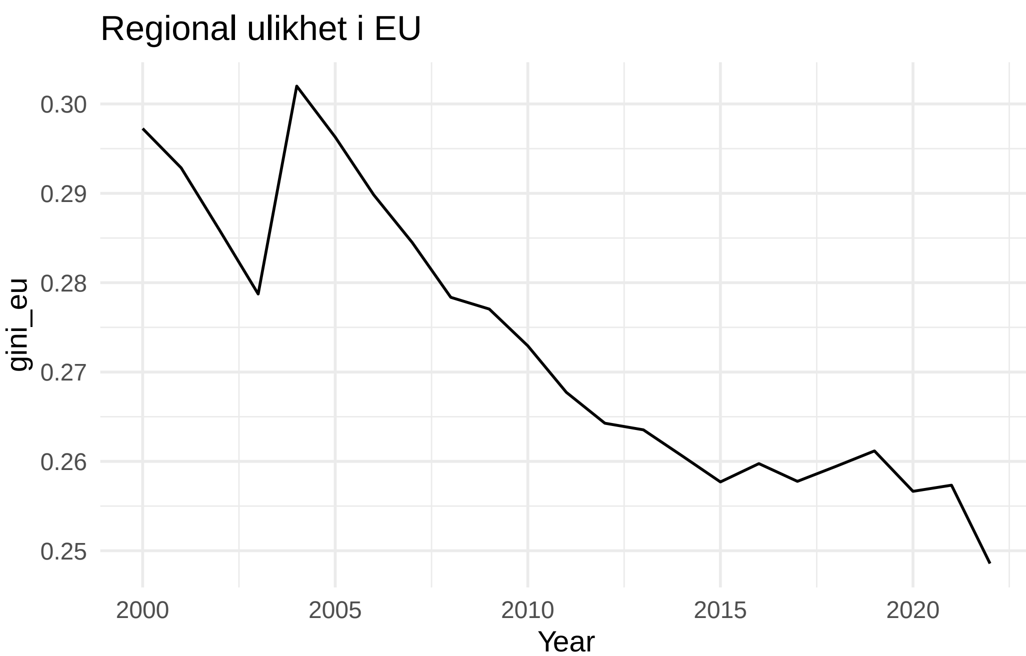

Figure 1: Utvikling i Gini-koeffisienten for NUTS3-regioner i EU.

Oppgave 19

Gini-koeffisienten mellom NUTS3-regioner viser en tydelig fallende trend over tid, slik Figure 1 illustrerer. Dette innebærer at regionale forskjeller gradvis reduseres. En slik utvikling er i tråd med målet om å styrke mindre utviklede områder gjennom EUs strukturfond. Figure 1 indikerer også at tiltaket kan ha ønsket effekt.

Oppgave 20

```
# Kombinerer
eu_data_nested <- gini_n3_nest |>
  left_join(gini_NUTS2_nest, by = c("nc", "nc_name")) |>
  left_join(gini_NUTS1_nest, by = c("nc", "nc_name")) |>
  left_join(gini_NUTSc_nest, by = c("nc", "nc_name"))
```

Plots som viser utviklingen

Oppgave 21

```
gini_nc |>
  filter(!is.na(gini_nc)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(
    x = time,
    y = gini_nc,
    group = nc_name,
    color = nc_name
  )) +
  geom_line() +
  theme_minimal() +
  labs(
    title = "Regional ulikhet over tid",
    x = "time",
    y = "gini_nc",
    color = "nc_name"
  )
```

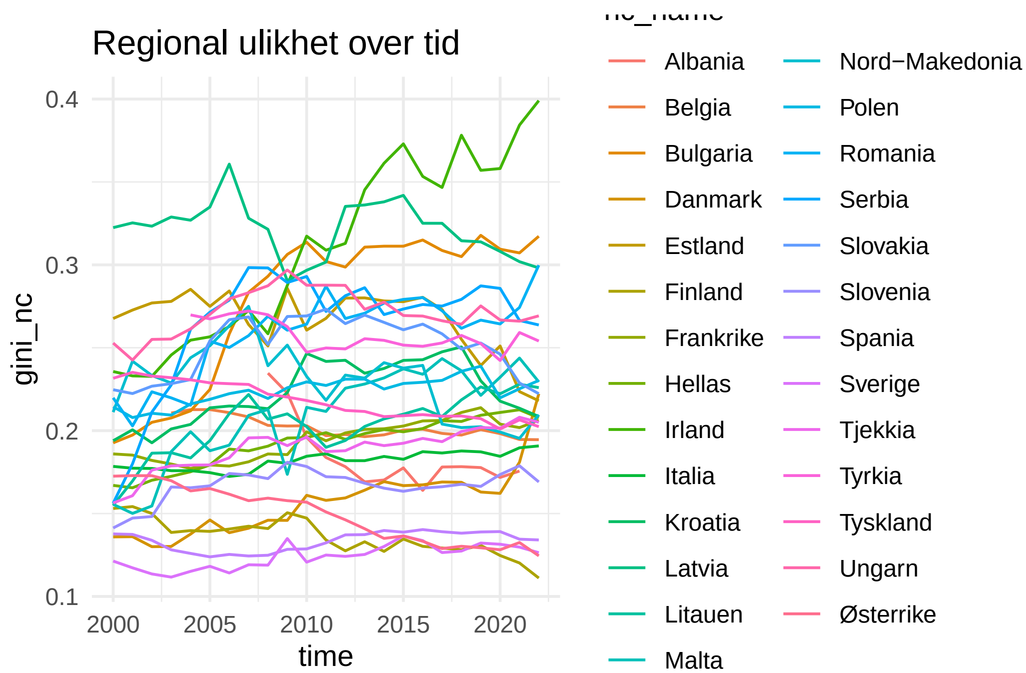

Figure 2: Utviklingen over tid for Gini-koeffisienten på nasjonsnivå for de 29 landene. Gini-koeffisienten måler hvordan verdiskapingen i landet er fordelt mellom NUTS3-regioner.

Oppgave 22

```r
gini_nc |>
  filter(time == 2022) |>
  arrange(desc(gini_nc)) |>
  select(nc_name, gini_nc) |>
  knitr::kable(digits = 7)
```

Table 10: Gini-koeffisient for BNP i 2022. Kypros og Luxembourg har bare en NUTS3-region og dermed ingen Gini-koeffisient.

nc_name	gini_nc
Irland	0.3990713
Bulgaria	0.3172612
Romania	0.2998082
Latvia	0.2981280
Ungarn	0.2692496

Table 10: Gini-koeffisient for BNP i 2022. Kypros og Luxembourg har bare en NUTS3-region og dermed ingen Gini-koeffisient.

nc_name	gini_nc
Serbia	0.2637424
Tyrkia	0.2540632
Polen	0.2302756
Malta	0.2294089
Litauen	0.2259366
Slovakia	0.2222795
Danmark	0.2222566
Estland	0.2180376
Nord-Makedonia	0.2095793
Kroatia	0.2085360
Hellas	0.2077325
Frankrike	0.2062529
Tjekkia	0.2050454
Tyskland	0.2024255
Belgia	0.1945446
Italia	0.1907755
Slovenia	0.1690681
Spania	0.1340889
Sverige	0.1263810
Østerrike	0.1244962
Finland	0.1111034
Kypros	NaN
Luxembourg	NaN

Oppgave 23

```
gini_series <- gini_nc |>
  transmute(
    country = nc_name,
    year    = as.integer(time),
    gini    = gini_nc
  ) |>
  filter(!is.na(gini), !is.nan(gini)) |>
  arrange(country, year) |>
  group_by(country) |>
  mutate(
```

```r
    gini_first = first(gini),
    year_comp  = if (any(year == 2022L)) 2022L else last(year),
    gini_comp  = if (any(year == 2022L)) gini[year == 2022L][1] else last(gini),
    group      = if_else(gini_comp < gini_first, "Lower", "Higher")
  ) |>
  ungroup()
```

```r
gini_series |>
  filter(group == "Lower") |>
  ggplot(aes(year, gini, colour = country, group = country)) +
  geom_line() +
  labs(title = "Lavere regional ulikhet", x = "Year", y = "Gini-koeffisient", colour = "Land"
  ) +
  theme_minimal()
```

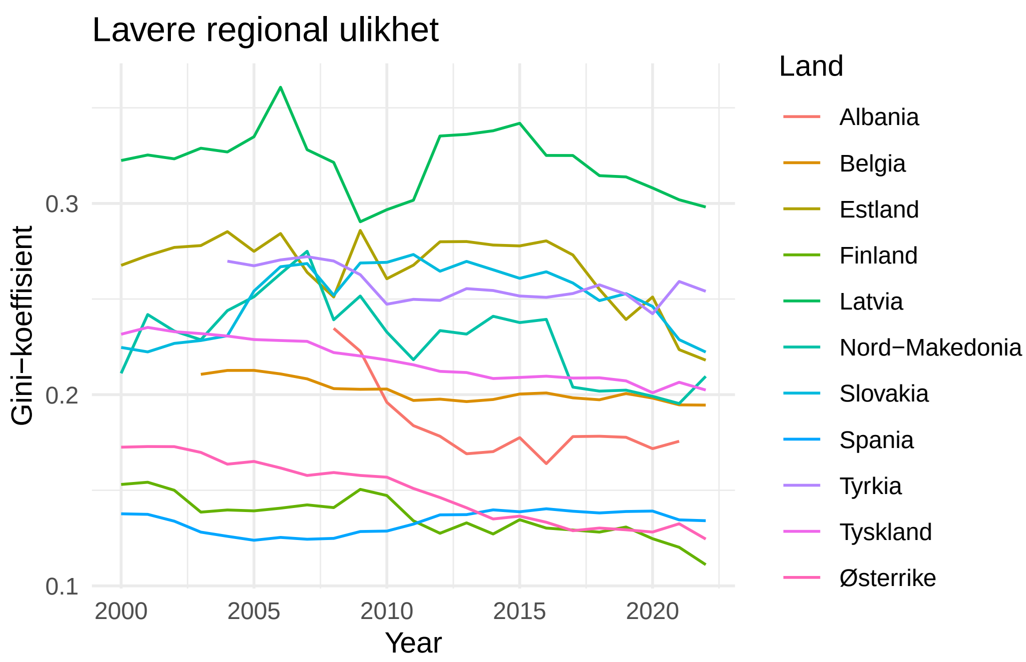

Figure 3: Land med lavere regional ulikhet i 2022 enn første år vi har data for.

```r
gini_series |>
  filter(group == "Higher") |>
  ggplot(aes(year, gini, colour = country, group = country)) +
  geom_line() +
  labs(title = "Høyere regional ulikhet", x = "Year", y = "Gini-koeffisient", colour = "Land"
```

```
  ) +
  theme_minimal()
```

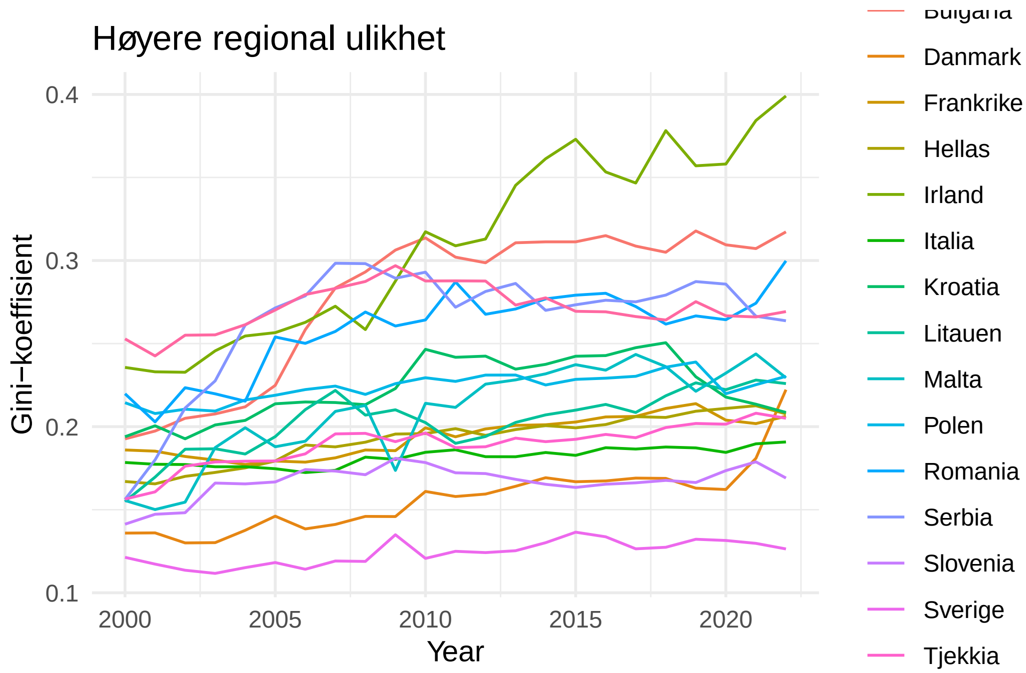

Figure 4: Land med høyere regional ulikhet i 2022 enn første år vi har data for.

Oppgave 24

```
gini_n2 |>
  filter(nc == "IE", !is.na(gini_n2)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(x = time, y = gini_n2, group = n2, color = n2)) +
  geom_line() +
  scale_x_continuous(
    breaks = seq(2000, 2022, by = 5)
  ) +
  theme_minimal() +
  labs(
    title = "Regional ulikhet i Irland",
    x = "Year",
    y = "gini_n2",
```

```
    color = "n2"
  )
```

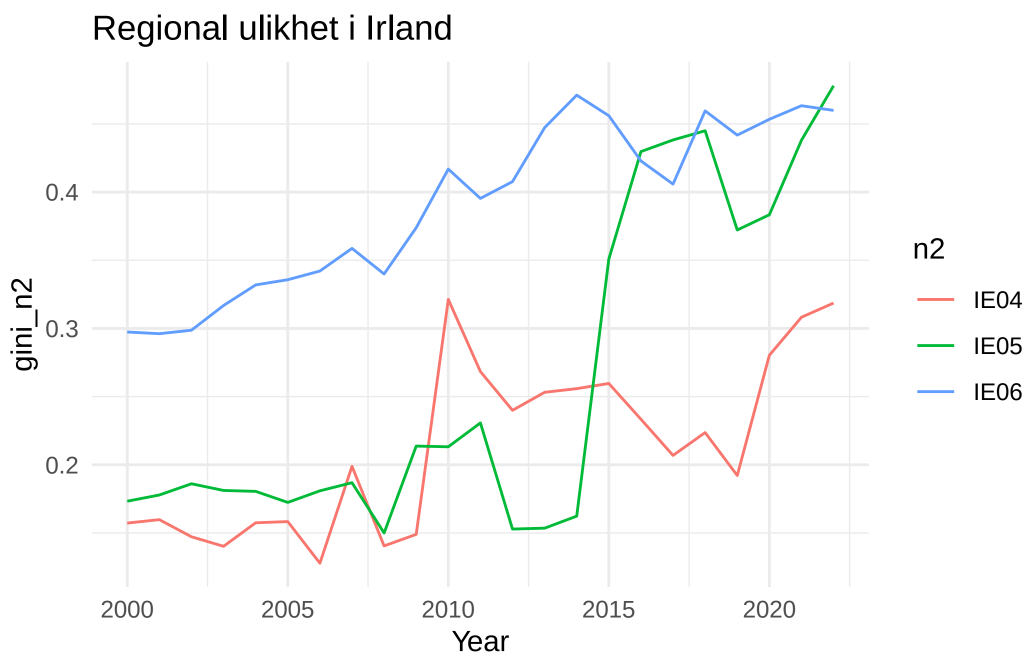

Figure 5: Utviklingen i Gini-koeffisienten for NUTS2-regionene i Irland.

Hvordan er verdiskapningen fordelt mellom regionene i ulike land?

Spania

Oppgave 25

```
gini_n2 |>
  filter(nc == "ES", !is.na(gini_n2)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(x = time, y = gini_n2, group = n2, colour = n2)) +
  geom_line() +
  scale_x_continuous(breaks = seq(2000, 2022, by = 5)) +
  labs(
    title = "Regional (NUTS2) ulikhet i Spania",
    x = "Year",
```

```
    y = "gini_n2",
    colour = "n2"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom"
  )
```

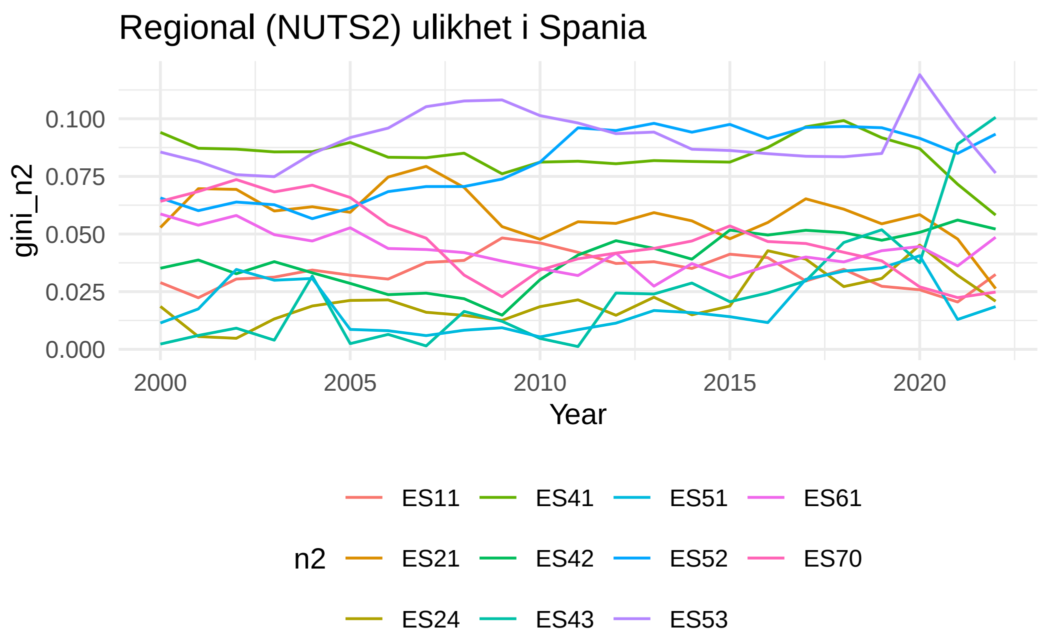

Figure 6: Utviklingen i Gini-koeffisienten for NUTS2-regionene i Spania.

Oppgave 26

```
gini_n1 |>
  filter(nc == "ES", !is.na(gini_n1)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(x = time, y = gini_n1, group = n1, colour = n1)) +
  geom_line() +
  scale_x_continuous(breaks = seq(2000, 2022, by = 5)) +
  labs(
    title  = "Regional (NUTS1) ulikhet i Spania",
```

```
    x      = "Year",
    y      = "gini_n1",
    colour = "n1"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```

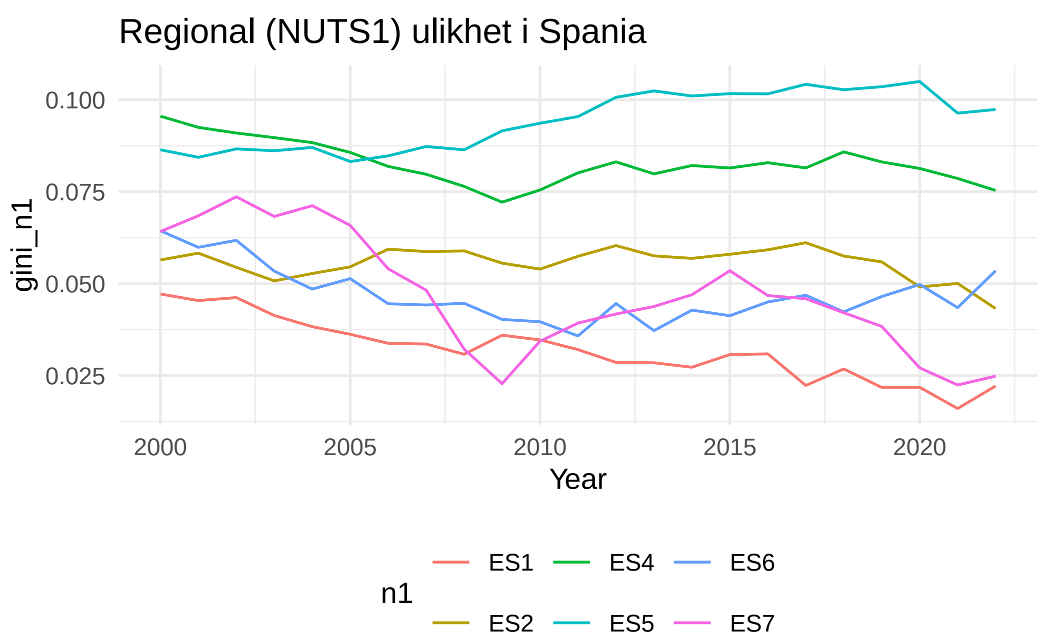

Figure 7: Utviklingen i Gini-koeffisienten for NUTS1-regionene i Spania.

NUTS1-regionene som har hatt økt utjevning kjennetegnes av en fallende eller stabil Gini-koeffisient over tid. Dette indikerer mindre forskjeller i verdiskaping mellom regionene innen disse områdene. Samtidig varierer utviklingen mellom regionene, og figuren alene gir ikke grunnlag for å trekke sterke konklusjoner om årsakene.

Tyskland

Oppgave 27

```
gini_n2 |>
  filter(nc == "DE", !is.na(gini_n2)) |>
```

```
  mutate(time = as.integer(time)) |>
  ggplot(aes(x = time, y = gini_n2, group = n2)) +
  geom_line(colour = "black", alpha = 0.7) +
  scale_x_continuous(breaks = seq(2000, 2022, by = 5)) +
  labs(
    title = "Regional (NUTS2) ulikhet i Tyskland",
    x = "Year",
    y = "gini_n2"
  ) +
  theme_minimal()
```

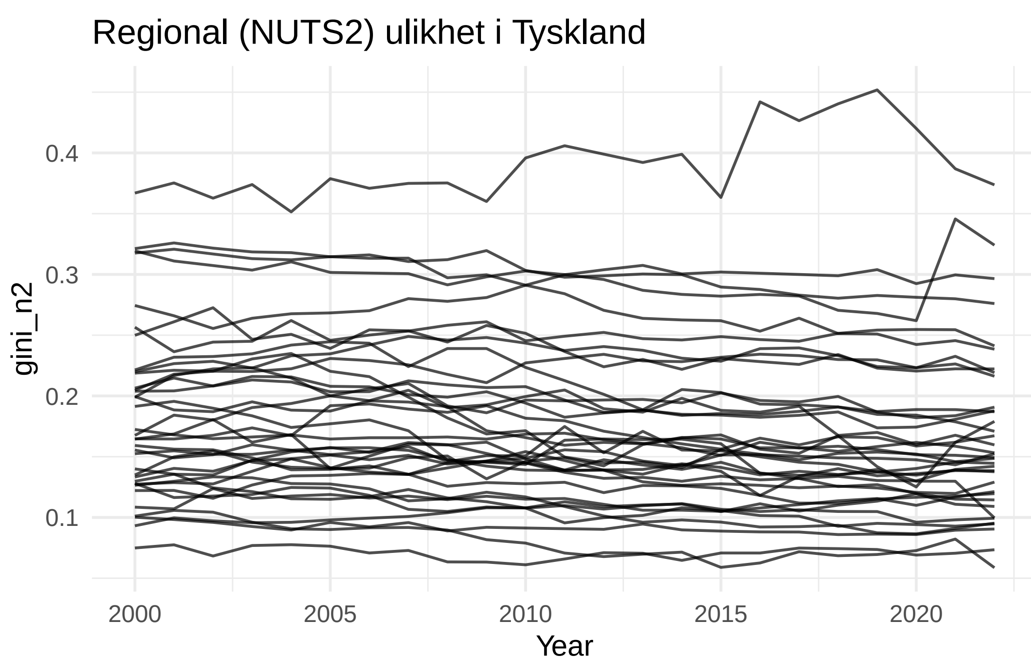

Figure 8: Utviklingen i Gini-koeffisienten for NUTS2-regionene i Tyskland.

```
# Finner NUTS2-regionen i Tyskland med høyest observert Gini-koeffisient.

gini_n2 |>
  filter(nc == "DE", !is.na(gini_n2)) |>
  group_by(n2) |>
  summarise(
    max_gini = max(gini_n2),
    .groups = "drop"
  ) |>
```

```
  arrange(desc(max_gini)) |>
  slice(1)
```

```
# A tibble: 1 x 2
  n2    max_gini
  <chr>    <dbl>
1 DE91     0.452
```

Fra Figure 8 ser vi at Gini-koeffisientene for NUTS2-regionene i Tyskland i analyseperioden varierer omtrent fra 0,03 til over 0,45, noe som indikerer betydelige forskjeller mellom regionene. Enkelte NUTS2-regioner har gjennom hele perioden Gini-koeffisienter som ligger klart høyere enn øvrige regioner, noe som tyder på større interne regionale forskjeller, mens andre regioner over tid har hatt gjennomgående lavere nivåer.

Basert på dataene kan regionen med de største observerte regionale forskjellene identifiseres som NUTS2-regionen DE91, som ifølge Wikipedia: NUTS statistical regions of Germany tilsvarer Braunschweig. Denne geografiske plasseringen kan imidlertid ikke leses direkte ut av figuren alene, men fremkommer først når NUTS-kodene kobles til tilhørende regioninformasjon.

Oppgave 28

```
gini_n1 |>
  filter(nc == "DE", !is.na(gini_n1)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(
    x = time,
    y = gini_n1,
    group = n1,
    colour = n1
  )) +
  geom_line() +
  scale_x_continuous(breaks = seq(2000, 2022, by = 5)) +
  labs(
    title  = "Regional (NUTS1) ulikhet i Tyskland",
    x      = "Year",
    y      = "Gini-koeffisient",
    colour = "n1"
  ) +
  theme_minimal() +
  theme(
```

```
  legend.position = "right"
)
```

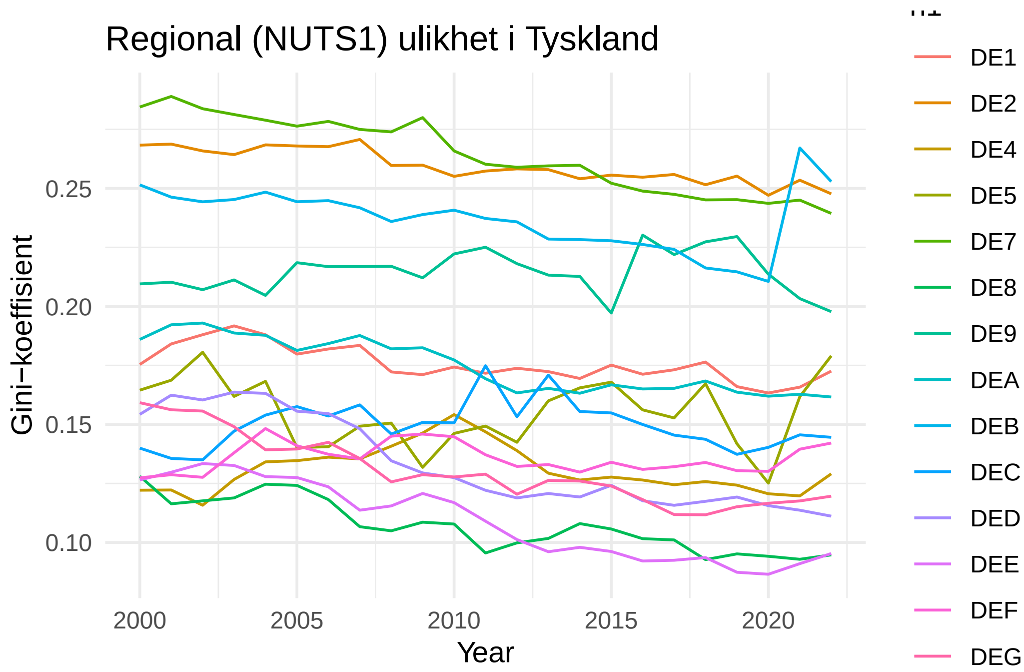

Figure 9: Utviklingen i Gini-koeffisienten for NUTS1-regionene i Tyskland.

```
eu_data_nested |>
  unnest(NUTS1_data) |>
  filter(nc_name == "Tyskland") |>
  filter(time == "2022") |>
  select(n1, gini_n1, num_reg_n1) |>
  arrange(desc(gini_n1)) |>
  flextable() |>
  line_spacing(space = 0.3) |>
  colformat_double(j = 2, digits = 4)
```

n1	gini_n1	num_reg_n1
DEB	0.2529	36
DE2	0.2477	96
DE7	0.2394	26

n1	gini_n1	num_reg_n1
DE9	0.1978	45
DE5	0.1790	2
DE1	0.1726	44
DEA	0.1616	53
DEC	0.1445	6
DEF	0.1421	15
DE4	0.1291	18
DEG	0.1196	22
DED	0.1111	13
DEE	0.0953	14
DE8	0.0947	8
DE3		1
DE6		1

Frakrike

Oppgave 29

```
gini_n1 |>
  filter(nc == "FR", !is.na(gini_n1)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(
    x = time,
    y = gini_n1,
    group = n1,
    colour = n1
  )) +
  geom_line() +
  scale_x_continuous(breaks = seq(2000, 2022, by = 5)) +
  labs(
    title  = "Regional (NUTS1) ulikhet i Frankrike",
    x      = "Year",
    y      = "gini_n1",
```

```r
    colour = "n1"
  ) +
  theme_minimal() +
  theme(
    legend.position = "right"
  )
```

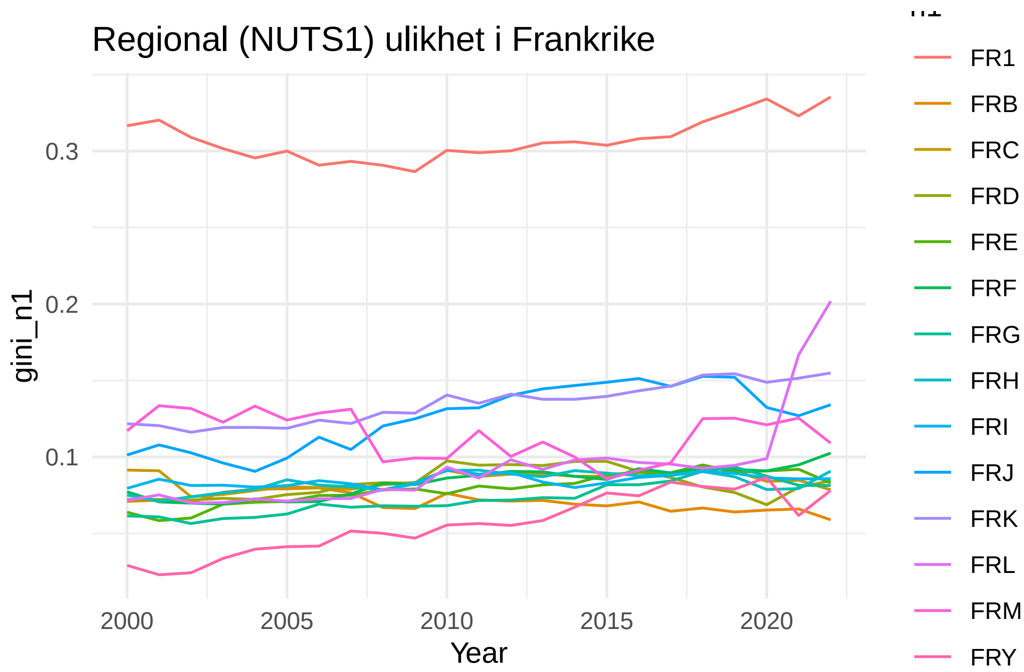

Figure 10: Utviklingen i Gini-koeffisienten for NUTS1-regionene i Frankrike.

Oppgave 30

```r
# Viser de seks NUTS1-regionene i Frankrike med høyest Gini-koeffisient i 2022.

top6_fr_n1_2022 <- gini_n1 |>
  filter(nc == "FR", time == 2022, !is.na(gini_n1)) |>
  arrange(desc(gini_n1)) |>
  select(n1, gini_n1) |>
  slice_head(n = 6)

top6_fr_n1_2022
```

```
# A tibble: 6 x 2
  n1    gini_n1
  <chr>   <dbl>
1 FR1     0.335
2 FRL     0.202
3 FRK     0.155
4 FRJ     0.134
5 FRM     0.109
6 FRF     0.103
```

Basert på tabellen ser vi at NUTS1-sonen FR1 har suverent høyest Gini-koeffisient i 2022, med en verdi på 0,335, som er klart høyere enn de øvrige regionene. Ifølge Wikipedia: NUTS statistical regions of France tilsvarer FR1 regionen Île-de-France, som ligger i og rundt Paris-området. Dette innebærer at de største regionale forskjellene i Frankrike i 2022 er konsentrert i hovedstadsregionen.

Oppgave 31

```r
eu_data |>
  filter(nc == "FR", n1 == "FR1", !is.na(gdp_pc_n3)) |>
  mutate(time = as.integer(time)) |>
  ggplot(aes(
    x = time,
    y = gdp_pc_n3,
    group = n3,
    colour = n3
  )) +
  geom_line() +
  theme_minimal() +
  labs(
    title = "BNP per innbygger i NUTS3-regionene i Île-de-France",
    x = "År",
    y = "GDP per capita",
    colour = "NUTS3"
  )
```

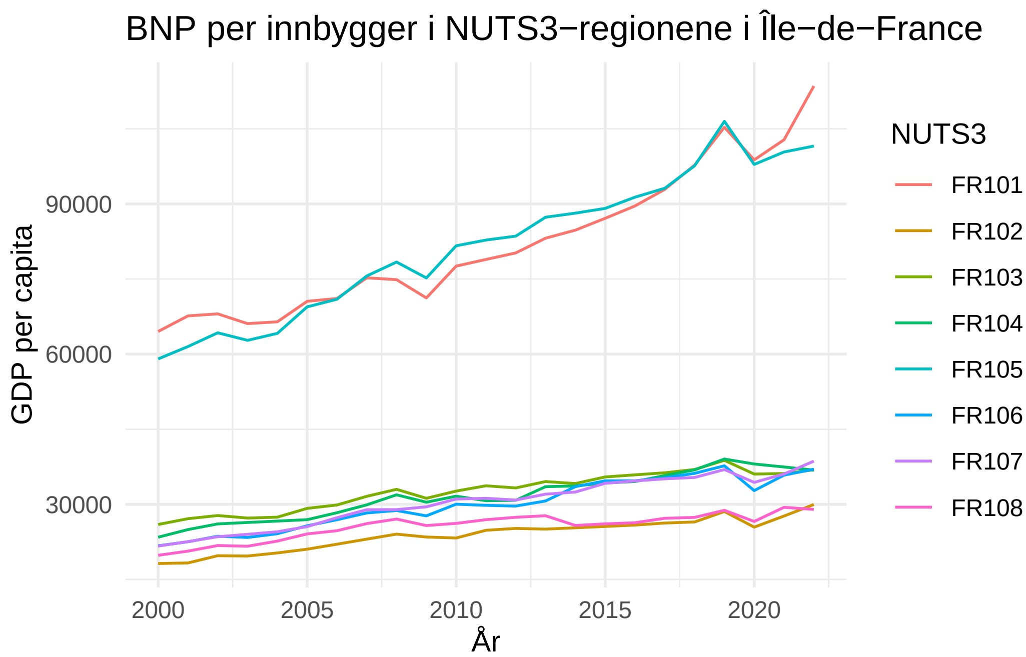

Figure 11: BNP per innbygger i NUTS3-regionene i Île-de-France.

```
# Viser BNP per innbygger i 2022 for NUTS3-regionene i FR1.
eu_data |>
  filter(nc == "FR", n1 == "FR1", time == 2022, !is.na(gdp_pc_n3)) |>
  arrange(desc(gdp_pc_n3)) |>
  select(n3, gdp_pc_n3)
```

```
# A tibble: 8 x 2
  n3    gdp_pc_n3
  <chr>     <dbl>
1 FR101   113523.
2 FR105   101546.
3 FR107    38625.
4 FR106    37017.
5 FR103    36897.
6 FR104    36840.
7 FR102    29967.
8 FR108    28995.
```

Oppgave 32

Ja. Ut fra Figure 11 kan vi se at FR1 (Île-de-France) har betydelige interne forskjeller i BNP per innbygger mellom NUTS3-regionene. I 2022 ligger Paris (FR101) og Hauts-de-Seine (FR105) på et svært høyt nivå, med BNP per innbygger på om lag 100 000–115 000, mens de øvrige NUTS3-regionene i FR1 i hovedsak ligger i intervallet rundt 29 000–39 000. Denne tydelige konsentrasjonen av svært høye verdier i noen få NUTS3-regioner, kombinert med betydelig lavere nivåer i resten av regionen, bidrar til å trekke Gini-koeffisienten for FR1 opp.

Enkle modeller

Oppgave 33

```r
n3_data <- eu_data_nested |>
  unnest(NUTS3_data) |>
  select(nc_name, n2, n3, time, gdp_pc_n3)
```

```r
n2_data <- eu_data_nested |>
  unnest(NUTS2_data) |>
  select(nc_name, n2, time, gini_n2)
```

```r
NUTS2_diff <- n3_data |>
  left_join(n2_data, by = join_by(nc_name, n2, time)) |>
  mutate(
    diff_gdp_per_capita = c(NA, 100 * diff(gdp_pc_n3)),
    diff_gini_nuts2 = c(NA, 100 * diff(gini_n2))
  ) %>%
  filter(complete.cases(.)) |>
  group_by(nc_name, n2) |>
  nest(.key = "NUTS2_diff")
```

```r
unnest(NUTS2_diff, NUTS2_diff) |>
  ungroup() |>
  filter(!is.nan(gini_n2)) |>
  select(n2) |>
  distinct() |>
  nrow()
```

```
[1] 218
```

Oppgave 34

```r
NUTS2_diff <- NUTS2_diff  |>
  group_by(nc_name, n2) |>
  mutate(
    modell = map(
      .x = NUTS2_diff,
      .f = function(a_df) lm('diff_gini_nuts2 ~ diff_gdp_per_capita', data = a_df)
    )
  )
```

Oppgave 35

```r
NUTS2_diff <- NUTS2_diff |>
  group_by(nc_name, n2) |>
  mutate(
    mod_coeff = bind_rows(
      map(.x = modell, .f = coef)
    )
  )
```

Oppgave 36

```r
NUTS2_diff <- NUTS2_diff  |>
  group_by(nc_name, n2) |>
  mutate(
    mod_sum = bind_rows(
      map(
        .x = modell,
        .f = glance
        )
    )
    )
```

Oppgave 37

```r
# Viser de tre NUTS2-regionene med høyest R² i modellen

top3_r2 <- NUTS2_diff |>
  ungroup() |>
  tidyr::unnest(mod_sum) |>
  filter(!is.na(r.squared), !is.nan(r.squared)) |>
  arrange(desc(r.squared), nc_name, n2) |>
  select(nc_name, n2, NUTS2_diff, modell, mod_coeff, r.squared) |>
  slice(1:3)
top3_r2
```

```
# A tibble: 3 x 6
  nc_name  n2    NUTS2_diff        modell mod_coeff$`(Intercept)` r.squared
  <chr>    <chr> <list>            <list>                   <dbl>     <dbl>
1 Polen    PL82  <tibble [92 x 6]> <lm>                   -0.0364     0.772
2 Tyskland DED5  <tibble [69 x 6]> <lm>                    0.00344    0.701
3 Belgia   BE22  <tibble [60 x 6]> <lm>                    0.129      0.692
# i 1 more variable: mod_coeff$diff_gdp_per_capita <dbl>
```

```r
# Viser de tre NUTS2-regionene med lavest R² i modellen

bot3_r2 <- NUTS2_diff |>
  ungroup() |>
  tidyr::unnest(mod_sum) |>
  filter(!is.na(r.squared), !is.nan(r.squared)) |>
  arrange(r.squared, nc_name, n2) |>
  select(nc_name, n2, NUTS2_diff, modell, mod_coeff, r.squared) |>
  slice(1:3)
bot3_r2
```

```
# A tibble: 3 x 6
  nc_name  n2    NUTS2_diff         modell mod_coeff$`(Intercept)` r.squared
  <chr>    <chr> <list>             <list>                   <dbl>     <dbl>
1 Italia   ITH3  <tibble [160 x 6]> <lm>                    0.00374  0.000102
2 Tyskland DE26  <tibble [276 x 6]> <lm>                   -0.00903  0.000150
3 Tyskland DE22  <tibble [276 x 6]> <lm>                   -0.0500   0.000491
# i 1 more variable: mod_coeff$diff_gdp_per_capita <dbl>
```

Oppgave 38

```r
# Viser de tre NUTS2-regionene med høyest koeffisient for diff_gdp_per_capita

top3_coeff <- NUTS2_diff |>
  ungroup() |>
  mutate(coeff_diff_gdp = mod_coeff$diff_gdp_per_capita) |>
  filter(!is.na(coeff_diff_gdp), !is.nan(coeff_diff_gdp)) |>
  arrange(desc(coeff_diff_gdp), nc_name, n2) |>
  select(nc_name, n2, NUTS2_diff, modell, coeff_diff_gdp) |>
  slice(1:3)

top3_coeff
```

```
# A tibble: 3 x 5
  nc_name  n2    NUTS2_diff          modell coeff_diff_gdp
  <chr>    <chr> <list>              <list>          <dbl>
1 Bulgaria BG34  <tibble [92 x 6]>  <lm>        0.0000122
2 Bulgaria BG42  <tibble [115 x 6]> <lm>        0.0000111
3 Tyrkia   TRA2  <tibble [76 x 6]>  <lm>        0.0000101
```

Oppgave 39

```r
# Teller hvor mange koeffisienter for diff_gdp_per_capita som er signifikante på 5%-nivå

signifikante_koeff <- NUTS2_diff |>
  ungroup() |>
  tidyr::unnest(mod_sum) |>
  filter(!is.na(p.value), !is.nan(p.value)) |>
  filter(p.value < 0.05) |>
  nrow()

signifikante_koeff
```

```
[1] 162
```

162 av de 218 estimerte koeffisientene er signifikante på 5 % nivå.

Oppgave 40

```r
# Slår sammen NUTS2-nivådata
d <- NUTS2_diff |> tidyr::unnest(NUTS2_diff)

# Beregner 99,5-persentilen av absoluttverdiene til diff_gdp_per_capita
q <- quantile(abs(d$diff_gdp_per_capita), 0.995, na.rm = TRUE)

# Definerer en skaleringsfaktor
scale_factor <- q / 1e-05

# Skalerer diff_gdp_per_capita ved å dele på skaleringsfaktoren
d <- d |> mutate(diff_gdp_scaled = diff_gdp_per_capita / scale_factor)
```

```r
ggplot(d, aes(x = diff_gdp_scaled)) +
  geom_density(
    na.rm = TRUE,
    color = "black",
    linewidth = 1,
    bw = 2e-06
  ) +
  geom_vline(
    xintercept = mean(d$diff_gdp_scaled, na.rm = TRUE),
    linetype = "dashed",
    color = "grey40"
  ) +
  coord_cartesian(xlim = c(-2e-05, 1e-05)) +
  labs(
    title = "Tetthetsfordeling av diff_gdp_per_capita",
    x = "diff_gdp_per_capita",
    y = "density"
  ) +
  theme_grey()
```

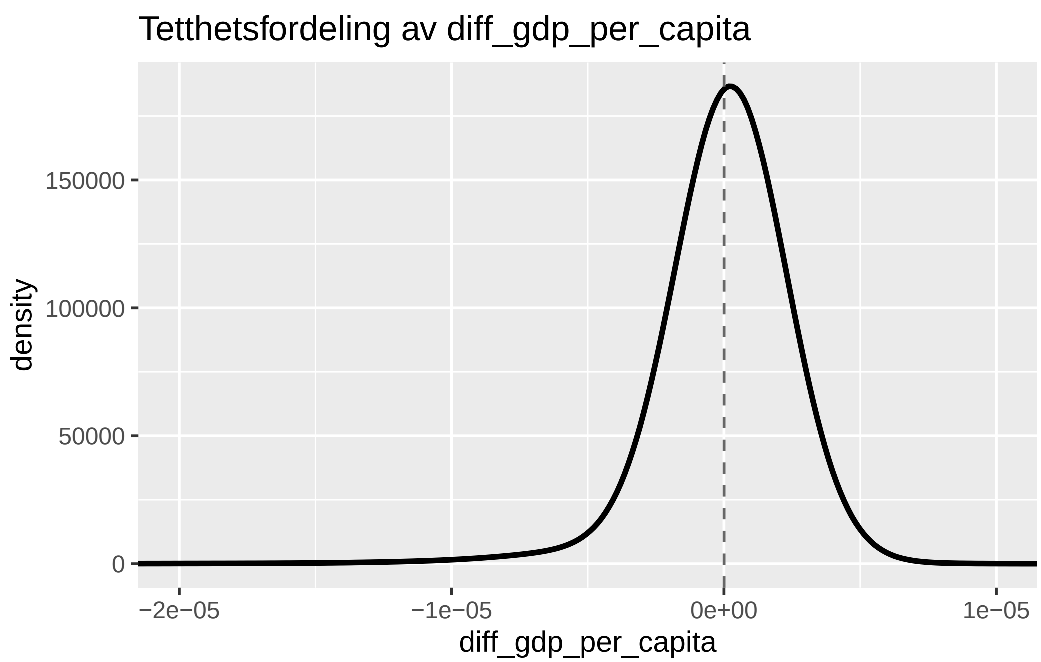

Figure 12: Tetthetsplott av diff_gdp_per_capita med stiplet linje for gjennomsnittet.

Oppgave 41

```
# Teller hvor mange regresjonskoeffisienter for diff_gdp_per_capita som er positive

positive_coeff <- NUTS2_diff |>
  ungroup() |>
  mutate(coeff_diff_gdp = mod_coeff$diff_gdp_per_capita) |>
  filter(!is.na(coeff_diff_gdp)) |>
  summarise(
    totalt = n(),
    positive = sum(coeff_diff_gdp > 0)
  )

positive_coeff
```

```
# A tibble: 1 x 2
  totalt positive
   <int>    <int>
1    218      132
```

132 av de 218 estimerte regresjonskoeffisientene for diff_gdp_per_capita er X positive.

Oppgave 42

```
NUTS2_diff |>
  ungroup() |>
  mutate(coeff_diff_gdp = mod_coeff$diff_gdp_per_capita) |>
  filter(!is.na(coeff_diff_gdp)) |>
  summarise(
    mean_coeff = mean(coeff_diff_gdp),
    median_coeff = median(coeff_diff_gdp)
  )
```

```
# A tibble: 1 x 2
   mean_coeff median_coeff
        <dbl>        <dbl>
1 0.000000816  0.000000528
```

Oppgave 43

```
# Utfører en ensidig t-test for å teste om gjennomsnittet av
# diff_gdp_per_capita er signifikant større enn 0

t_test_diff <- t.test(
  NUTS2_diff$mod_coeff$diff_gdp_per_capita,
  alternative = "greater",
  mu = 0
)

t_test_diff
```

```

	One Sample t-test

data:  NUTS2_diff$mod_coeff$diff_gdp_per_capita
t = 3.7658, df = 217, p-value = 0.0001069
alternative hypothesis: true mean is greater than 0
95 percent confidence interval:
```

```
 4.583272e-07          Inf
sample estimates:
  mean of x
8.16482e-07
```

Ja, diff_gdp_per_capita er signifikant større enn 0.

Panel modell

Oppgave 44

```r
d_panel <- d %>%
  ungroup() %>%
  select(n3, time, diff_gini_nuts2, diff_gdp_per_capita) %>%
  filter(!is.na(n3), !is.na(time))

p_mod <- plm(
  diff_gini_nuts2 ~ diff_gdp_per_capita,
  data  = d_panel,
  index = c("n3", "time")
)
```

Oppgave 45

```r
summary(p_mod)
```

```
Oneway (individual) effect Within Model

Call:
plm(formula = diff_gini_nuts2 ~ diff_gdp_per_capita, data = d_panel,
    index = c("n3", "time"))

Unbalanced Panel: n = 1186, T = 13-23, N = 26700

Residuals:
      Min.    1st Qu.     Median    3rd Qu.       Max.
-37.335219  -0.559292  -0.035798   0.505712  27.300139
```

```
Coefficients:
                    Estimate Std. Error t-value  Pr(>|t|)
diff_gdp_per_capita 3.9320e-07 2.4694e-08  15.923 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Total Sum of Squares:    78060
Residual Sum of Squares: 77291
R-Squared:      0.0098397
Adj. R-Squared: -0.036189
F-statistic: 253.536 on 1 and 25513 DF, p-value: < 2.22e-16
```

Oppgave 46

```r
summary(
  p_mod,
  vcov = function(x) plm::vcovHC(x, method = "white2")
)
```

```
Oneway (individual) effect Within Model

Note: Coefficient variance-covariance matrix supplied: function(x) plm::vcovHC(x, method = "w

Call:
plm(formula = diff_gini_nuts2 ~ diff_gdp_per_capita, data = d_panel,
    index = c("n3", "time"))

Unbalanced Panel: n = 1186, T = 13-23, N = 26700

Residuals:
      Min.    1st Qu.     Median    3rd Qu.       Max.
-37.335219  -0.559292  -0.035798   0.505712  27.300139

Coefficients:
                    Estimate Std. Error t-value  Pr(>|t|)
diff_gdp_per_capita 3.9320e-07 2.7778e-08  14.155 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Total Sum of Squares:    78060
Residual Sum of Squares: 77291
R-Squared:      0.0098397
Adj. R-Squared: -0.036189
F-statistic: 200.369 on 1 and 1185 DF, p-value: < 2.22e-16
```

Her benyttes en alternativ måte å generere summary() for panelmodellen ved å spesifisere en egen varians–kovariansmatrise for koeffisientene. Ved å bruke `vcovHC()` med metoden “white2” beregnes heteroskedastisitets-robuste standardfeil.

De to summary-ene er basert på samme panelregresjonsmodell og gir identiske koeffisientestimater, men standardfeilene og tilhørende teststatistikker avviker. I den ordinære summary()-en er standardfeilen 2.4694e−08 og t-verdien 15.923, mens den alternative summary()-en som benytter heteroskedastisitets-robuste standardfeil via `vcovHC()` gir en høyere standardfeil på 2.7778e−08 og en lavere t-verdi på 14.155. Tilsvarende er F-statistikken 253.536 (1 og 25513 frihetsgrader) i den ordinære summary()-en, mot 200.369 (1 og 1185 frihetsgrader) i den robuste versjonen. Forskjellene skyldes bruken av ulike varians–kovariansmatriser, mens den overordnede statistiske konklusjonen forblir uendret.